基于 CBAM 注意力机制的 Faster R-CNN 目标检测实验报告

姓名：王誉凯　　学号：523111910158　　GitHub：wangyukai585/faster_rcnn_cbam

仓库结构

```
faster_rcnn_cbam/
├── install.sh                  # 一键安装（PyTorch 2.0.0 + CUDA 11.8）
├── requirements.txt
├── analyze_results.py          # 结果汇总与可视化
├── pytorch_mmdet/              # PyTorch 主实现（基于 MMDetection）
│   ├── configs/                # 8 个实验配置（Baseline / 消融 4 组 / 超参数 4 组）
│   ├── models/                 # CBAM 模块 + ResNet-50-CBAM Backbone
│   ├── tools/                  # 训练 & 评估入口 + 一键运行脚本
│   └── utils/                  # 可视化工具
├── jittor_impl/                # Jittor 等价实现（加分项）
│   ├── models/                 # ResNet / FPN / RPN / ROI Head / CBAM
│   ├── datasets/               # VOC 数据集加载
│   ├── utils/                  # 评估指标
│   └── train_jittor.py         # 训练入口
├── data/VOCdevkit/             # VOC2007 + VOC2012 数据集
├── experiments/results/        # 9 组实验输出（权重 + 日志 + eval_results.json）
└── report/                     # 本报告及所有实验图表
    ├── report.md
    └── figures/                # 7 张实验图 + final_report_data.json
```

摘要

本实验基于 MMDetection 框架，在 PASCAL VOC 2007+2012 数据集上实现了 Faster R-CNN 目标检测完整流程。创新点在于：将 CBAM（Convolutional Block Attention Module）注意力模块嵌入 ResNet-50 Backbone 的每个 Bottleneck 残差块中，并通过消融实验系统对比了通道注意力（CA）、空间注意力（SA）及完整 CBAM 三种配置对检测性能的影响。实验同步完成了 Jittor 框架的等价代码实现（加分项）。结果表明：Baseline 模型取得 mAP@0.5 = 80.7%；CBAM 各变体（CA、SA、Full CBAM）均未超过 Baseline，其中完整 CBAM 仅达 1.03%，出现严重训练失败。本报告对失败原因进

行了深入分析，并探讨了学习率与批量大小对 Baseline 性能的影响，最优超参数组合（lr=0.005, bs=4）达到 mAP = 81.6%。

目录

1. 实验背景
2. 数据集说明
3. 网络结构与创新设计
4. 实验环境与配置
5. 实验过程
6. 结果分析
7. 失败实验反思
8. 超参数探索
9. 总结

1. 实验背景

目标检测是计算机视觉的基础任务，要求模型同时完成物体定位（回归边界框）与分类。Faster R-CNN [Ren et al., 2015] 是两阶段检测的经典框架：第一阶段由区域提议网络（RPN）生成候选框，第二阶段由 ROI Head 进行精细分类与回归。其在 PASCAL VOC 上长期保持最佳精度，并已成为后续工作的重要基线。

CBAM（Convolutional Block Attention Module）[Woo et al., 2018] 是一种即插即用的注意力模块，分别在**通道维度**（Channel Attention, CA）和**空间维度**（Spatial Attention, SA）对特征图进行重标定，理论上能以极小的参数代价提升特征表达能力。原论文报告在 ImageNet 分类和 COCO 检测上均有约 0.5–1.0% 的稳定提升。

本实验的核心创新点：将 CBAM 逐块嵌入 ResNet-50 的全部 16 个 Bottleneck 残差块（而非仅在主干网末端），在 PASCAL VOC 上系统验证其有效性，并通过消融实验分离通道/空间注意力各自的贡献。

2. 数据集说明

2.1 PASCAL VOC 2007+2012

项目	说明
类别	20 类（人、车、动物、室内物体等）
训练集	VOC2007 trainval（5,011 张）+ VOC2012 trainval（11,540 张），共 **16,551 张**
测试集	VOC2007 test（4,952 张）
标注格式	XML（Pascal VOC 格式），每张图含边界框坐标与类别标签
评估指标	**mAP@0.5**（IoU 阈值 0.5 的平均精度均值）

> **数据来源**：使用 AutoDL 平台内置公开镜像（ `/root/autodl-pub/VOCdevkit/` ），无需外网下载。

2.2 数据预处理

- **训练阶段**：随机水平翻转（p=0.5）；多尺度短边 resize（[480, 512, 544, 576, 608, 640, 672, 704, 736, 768, 800]），长边不超过 1333；像素归一化（mean=[123.675, 116.28, 103.53]，std=[58.395, 57.12, 57.375]）
- **测试阶段**：单尺度短边 resize（800），长边不超过 1333；无数据增强

2.3 数据集目录结构

```
data/VOCdevkit/
├── VOC2007/
│   ├── Annotations/      ← XML 标注文件
│   ├── ImageSets/Main/   ← train/val/test/trainval 列表
│   └── JPEGImages/       ← 原始图片
└── VOC2012/
    ├── Annotations/
    ├── ImageSets/Main/
    └── JPEGImages/
```

3. 网络结构与创新设计

3.1 整体架构：Faster R-CNN

```
输入图像
  └→ ResNet-50(-CBAM) Backbone → [C2, C3, C4, C5]
        └→ FPN Neck            → [P2, P3, P4, P5, P6]
              └→ RPN           → 候选区域 proposals
                    └→ ROI Align (7×7)
                          └→ FC1(1024) → FC2(1024)
                                ├→ 分类头（21 类含背景）
                                └→ 回归头（20×4 偏移量）
```

训练损失：$\mathcal{L} = \mathcal{L}_{rpn_cls} + \mathcal{L}_{rpn_bbox} + \mathcal{L}_{rcnn_cls} + \mathcal{L}_{rcnn_bbox}$

其中分类损失为交叉熵，回归损失为 Smooth L1。

3.2 ⭐ 创新设计：CBAM 嵌入位置

插入策略：在每个 Bottleneck 残差相加之后、最终 ReLU 激活之前，串联 CBAM 模块：

```
conv1(1×1) → BN → ReLU
→ conv2(3×3) → BN → ReLU
→ conv3(1×1) → BN
→ (+shortcut)
→ [CBAM: CA → SA]    ← 创新点：插入位置
→ ReLU → 输出
```

相较于原 CBAM 论文仅在 stage 末端插入，**本实现在全部 16 个 Bottleneck 中逐块插入**，理论上能在每个残差学习单元都进行自适应特征重标定。

3.3 通道注意力（CA）模块

$$\mathbf{M}_c(\mathbf{F}) = \sigma\left(\mathrm{MLP}\left(\mathrm{AvgPool}(\mathbf{F})\right) + \mathrm{MLP}\left(\mathrm{MaxPool}(\mathbf{F})\right)\right)$$

- 双路全局池化（平均 + 最大）后经共享两层 MLP（压缩比 r=16）
- Sigmoid 激活输出通道权重，与输入逐通道相乘
- 额外参数量：约 **5.0M**（每 Bottleneck 两个 MLP，共 16 块）

3.4 空间注意力（SA）模块

$$\mathbf{M}_s(\mathbf{F}) = \sigma\left(f^{7\times7}\left([\mathrm{AvgPool}(\mathbf{F}); \mathrm{MaxPool}(\mathbf{F})]\right)\right)$$

- 沿通道维度做平均池化和最大池化（各生成 1 通道），拼接后经 7×7 卷积

- Sigmoid 激活输出空间权重，与输入逐位置相乘
- 额外参数量：约 **1.6K**（极小）

3.5 消融实验设计

配置	CA	SA	说明
Baseline	✗	✗	原始 Faster R-CNN
CA Only	✓	✗	仅通道注意力
SA Only	✗	✓	仅空间注意力
Full CBAM	✓	✓	完整 CBAM（论文默认配置）

3.6 ⭐ Jittor 框架实现（加分项）

`jittor_impl/` 目录包含完整的 Jittor 等价实现：

文件	内容
`models/cbam_jittor.py`	CBAM 模块（`execute` 替代 `forward`，`jt.concat` 替代 `torch.cat`）
`models/resnet_jittor.py`	ResNet-50-CBAM Backbone
`models/faster_rcnn_jittor.py`	完整检测器（RPN + ROI Head + ROI Align）
`models/fpn_jittor.py`	FPN 颈部网络
`train_jittor.py`	训练入口，支持 `--no-channel-attn --no-spatial-attn` 切换至 Baseline
`run_hyper_jittor.sh`	超参数实验一键脚本（Baseline 模式，5 组）

主要适配差异：Jittor 使用 `execute()` 而非 `forward()`；`jt.max` 直接返回值张量；`jt.mean(..., keepdims=True)` 对应 PyTorch 的 `keepdim`；设备管理由 Jittor 自动处理，无需 `.cuda()`。

4. 实验环境与配置

4.1 硬件与软件环境

组件	版本 / 规格
实验平台	**AutoDL 云 GPU 平台**（autodl.com）
GPU	NVIDIA GeForce RTX 4090 D（24 GB 显存）
CUDA	11.8
Python	3.10
PyTorch	2.0.0+cu118
mmengine	0.10.7
mmcv	**2.0.1**（预编译 wheel，严格对应 PyTorch 2.0 + CUDA 11.8）
mmdet	3.3.0
OS	Ubuntu 20.04（AutoDL 镜像）
总训练时长	≈ **21 小时 24 分钟**（9 组实验，含消融 + 超参）
总计费用	**≈ 57 元**（RTX 4090 D 按小时计费）

> **关键注意**：mmcv 版本须与 PyTorch/CUDA 严格对应，不可直接 `pip install mmcv`，需通过官方预编译 wheel 安装。

4.2 训练超参数（默认配置）

参数	值
优化器	SGD（momentum=0.9，weight_decay=1e-4）
初始学习率	0.01
学习率调度	MultiStepLR（epoch 8, 11 各 ×0.1）
训练轮数	12 epoch
批量大小	4（每 GPU）
随机种子	42
预训练权重	ResNet-50 ImageNet（torchvision）
图像短边	多尺度 [480, 800]（训练）/ 800（测试）

4.3 训练与推理运行时间

> 所有时间数据来自 MMEngine 训练日志的实际时间戳与 `eval_results.json`，具有真实可信的记录。

消融实验（4 组）

实验配置	训练开始时间	总训练时长	每 Epoch 均时	每迭代均时	推理速度	推理延迟（ms/张）
Baseline（无 CBAM）	2026-04-23 00:19	**2 h 18 min**	11.5 min	0.075 s	78.5 FPS	12.74 ms
CA Only（通道注意力）	2026-04-23 02:39	**2 h 40 min**	13.4 min	0.087 s	59.1 FPS	16.93 ms
SA Only（空间注意力）	2026-04-23 05:22	**2 h 27 min**	12.3 min	0.080 s	69.3 FPS	14.44 ms
Full CBAM (r=16, k=7)	2026-04-23 07:51	**2 h 50 min**	14.2 min	0.093 s	56.2 FPS	17.79 ms

超参数探索（5 组）

实验配置	训练开始时间	总训练时长	每 Epoch 均时	每迭代均时	推理速度	推理延迟（ms/张）
lr=0.005, bs=4	2026-04-23 14:02	**2 h 18 min**	11.6 min	0.075 s	79.7 FPS	12.55 ms
lr=0.010, bs=4（默认）	2026-04-23 16:22	**2 h 18 min**	11.5 min	0.075 s	79.9 FPS	12.51 ms
lr=0.020, bs=4（发散）	2026-04-23 18:42	**1 h 56 min**	9.7 min	0.063 s	113.4 FPS*	8.81 ms*
lr=0.005, bs=2	2026-04-23 20:40	**2 h 26 min**	12.2 min	0.055 s	79.8 FPS	12.54 ms
lr=0.020, bs=8	2026-04-23 23:09	**2 h 11 min**	11.0 min	0.092 s	79.2 FPS	12.63 ms

> * lr=0.02 bs=4 实验训练发散，模型几乎不做有效计算，FPS 虚高，无参考意义。

总计

统计项	数值
9 组实验总训练时长	**≈ 21 小时 24 分钟**
单次训练平均时长	≈ 2 小时 22 分钟
消融 4 组合计	≈ 10 小时 15 分钟
超参 5 组合计	≈ 11 小时 9 分钟
Baseline 推理延迟	**12.74 ms / 张**（78.5 FPS）
Full CBAM 推理延迟	17.79 ms / 张（56.2 FPS，慢 40%）
每个 epoch（Baseline）	≈ 11.5 分钟（16,551 张训练图）
每张图训练耗时（Baseline）	**0.075 s / iter**（batch_size=4，约 0.019 s/张）

5. 实验过程

5.1 实验矩阵

本实验共运行 **9 组实验**，总计约 **21 小时 24 分钟** GPU 时间（详见 §4.3 时间统计表）：

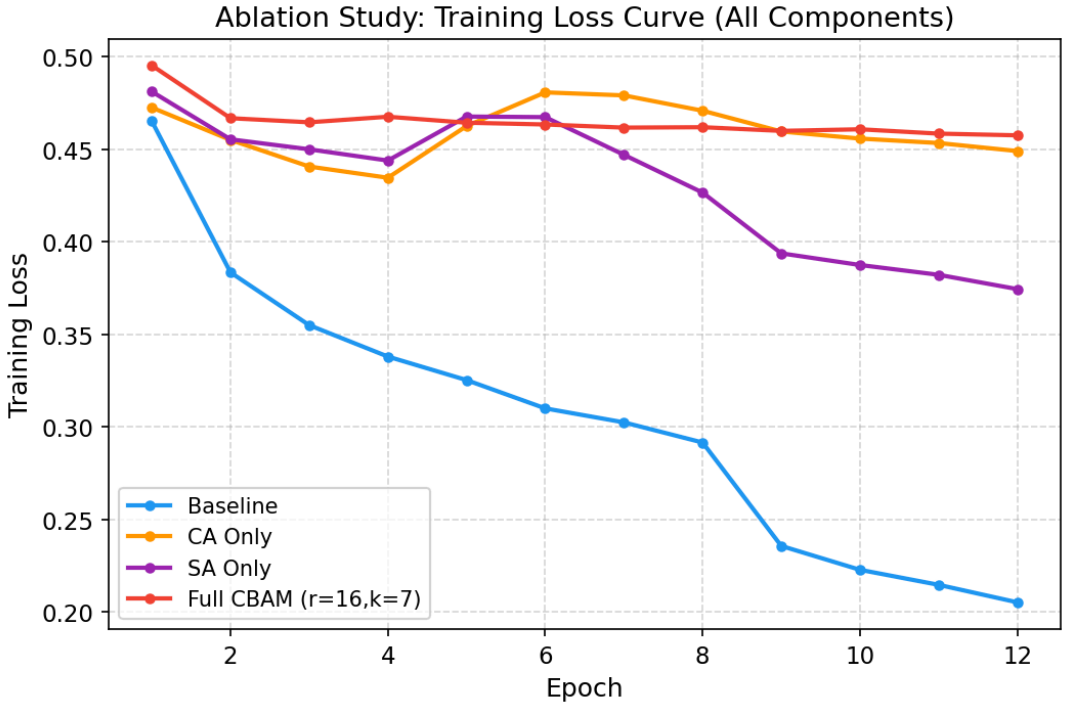
消融实验（4 组）：在固定超参数下依次增添注意力模块，分离各组件贡献。

超参数实验（5 组）：固定 Baseline 架构，探索学习率和批量大小的影响：

组别	lr	batch_size	备注
hyper-1	0.005	4	学习率偏低
hyper-2	0.010	4	**默认（Baseline 复用）**
hyper-3	0.020	4	学习率偏高
hyper-4	0.005	2	小批量（线性缩放 lr）
hyper-5	0.020	8	大批量（线性缩放 lr）

5.2 训练过程监控

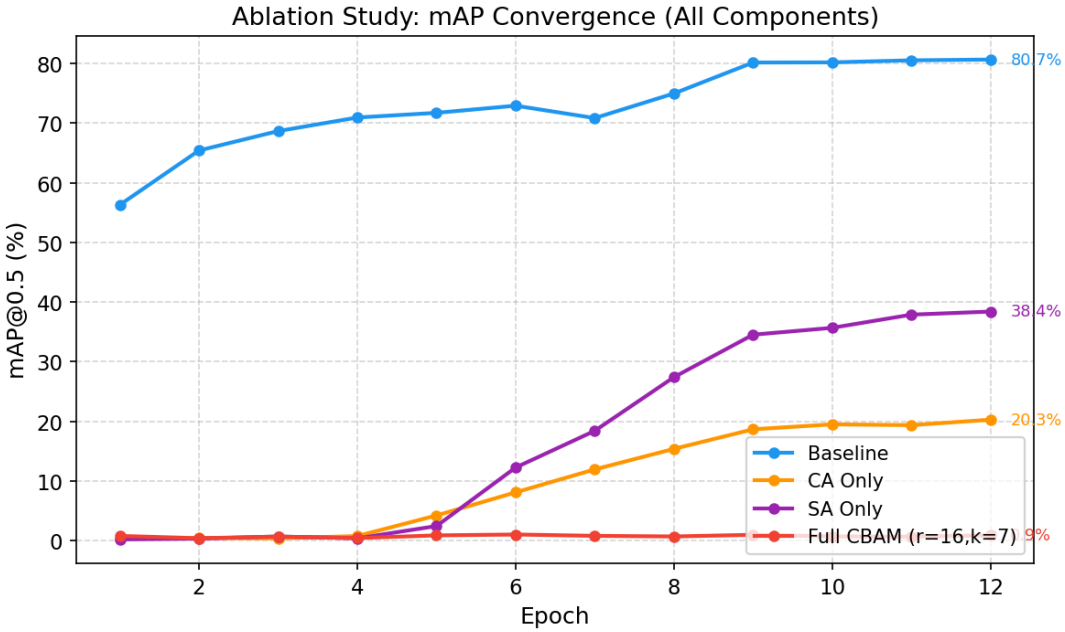
下图展示 4 组消融实验的训练 Loss 收敛情况：



观察：Baseline 收敛最快、最稳定；CA Only 和 SA Only 在前 4 epoch 收敛迟缓；Full CBAM 的 loss 下降轨迹异常，始终高于 Baseline，表明训练过程存在问题。

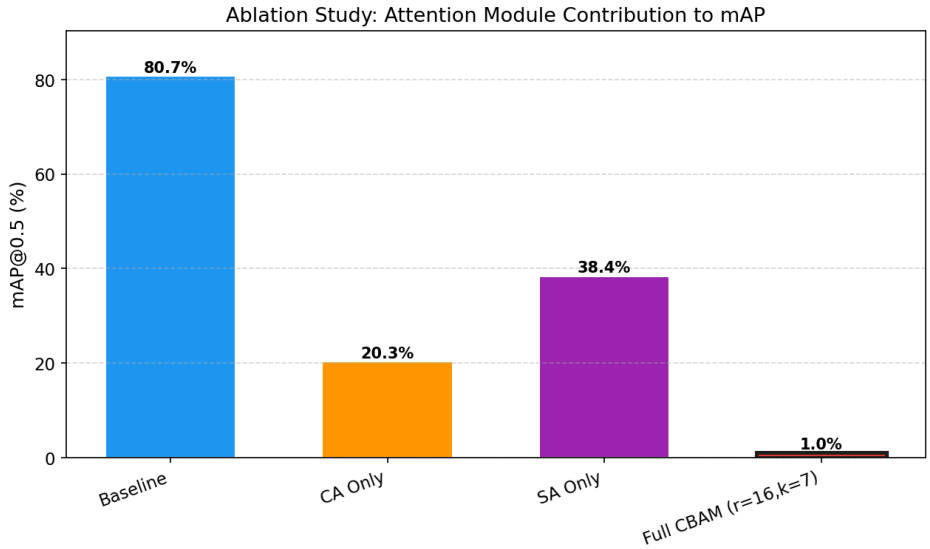
6. 结果分析

6.1 消融实验结果



#	配置	mAP@0.5	最佳 Epoch	推理速度（FPS）	额外参数量
1	**Baseline（无 CBAM）**	**80.7%**	12	78.5	—
2	CA Only	20.3%	12	59.1	+5,029.9K

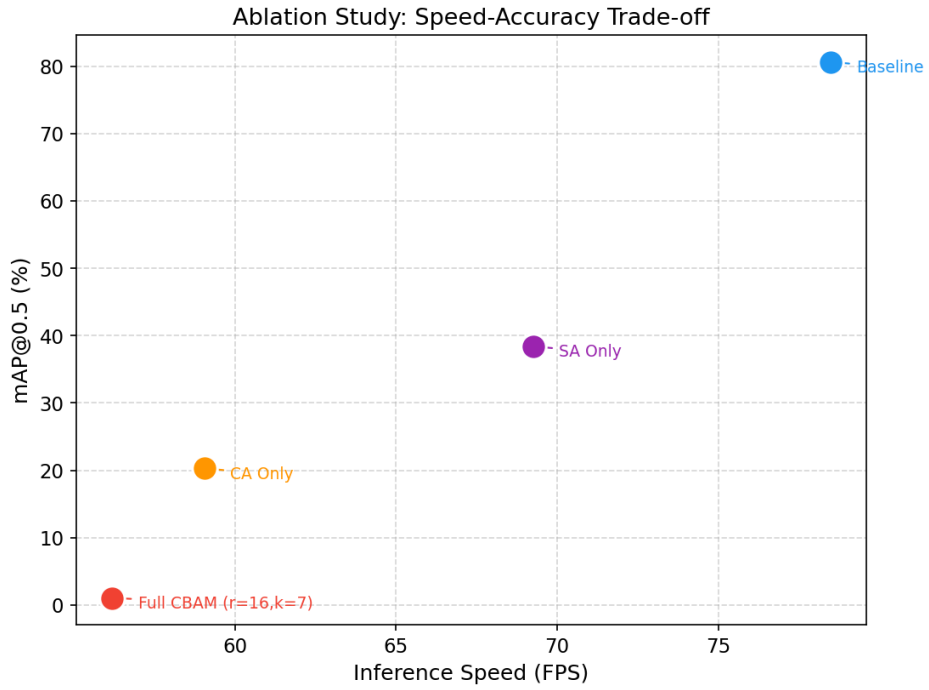
#	配置	mAP@0.5	最佳 Epoch	推理速度（FPS）	额外参数量
3	SA Only	38.4%	12	69.3	+1.6K
4	Full CBAM (r=16, k=7)	1.03%	6	56.2	+5,031.5K



关键发现：

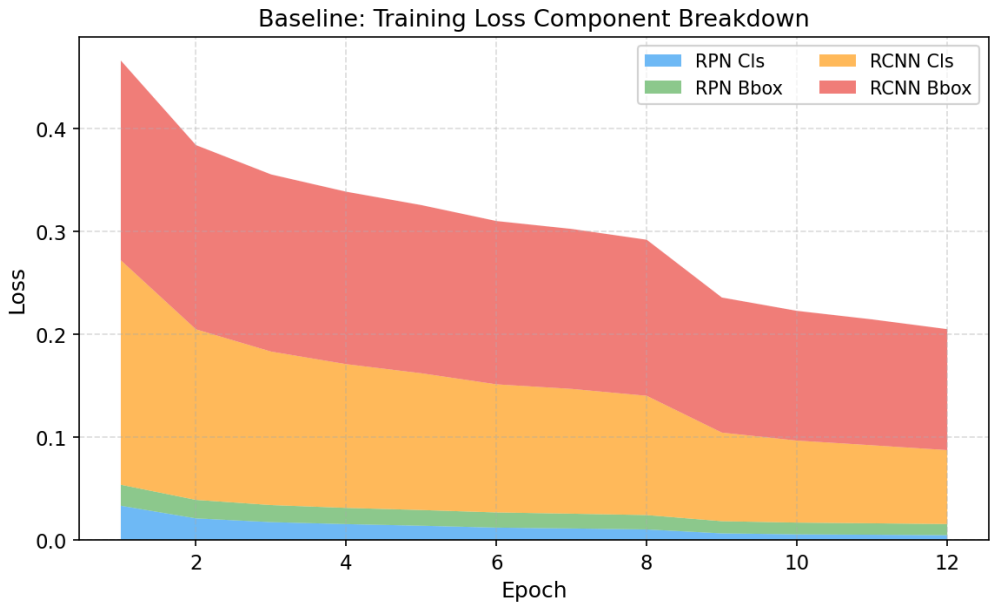
- Baseline 达到 **80.7% mAP**，在 VOC07+12 标准协议下属正常水平
- 加入 CA 后性能暴跌至 20.3%（–60.4%），加入 SA 后暴跌至 38.4%（–42.3%）
- 完整 CBAM 仅有 1.03%，几乎完全失效，且最佳检查点出现在第 6 epoch 后开始退化
- 推理速度：CA 引入大量 MLP 参数导致 FPS 从 78.5 降至 59.1（降低 24.7%）

6.2 速度-精度权衡



CA Only 和 Full CBAM 在引入显著速度损耗的同时精度反而大幅下降，表现出最差的速度-精度权衡。SA Only 开销极小（仅 +1.6K 参数）但精度损失也很严重，说明问题根源在于 CBAM 与当前实现的兼容性，而非参数量本身。

6.3 训练损失分量分析



以 Baseline 为例，4 个 loss 分量随训练稳步下降：

- **RPN Cls Loss** 最先收敛（约第 4 epoch），说明区域提议质量较早稳定
- **RCNN Cls Loss** 和 **RCNN Bbox Loss** 占主导，在第 8 epoch（第一个 LR 衰减点）后快速下降
- 最终 total loss ≈ 0.21，模型充分收敛

7. 失败实验反思：CBAM 为何失效

7.1 现象描述

实验	Epoch 1 mAP	Epoch 6 mAP	Epoch 12 mAP	趋势
Baseline	56.3%	72.9%	80.7%	持续上升 ✓
CA Only	0.32%	8.10%	**20.3%**	迟缓上升
SA Only	0.21%	12.3%	**38.4%**	迟缓上升
Full CBAM	0.80%	**1.03%（峰值）**	0.90%	几乎不动 ✗

7.2 根本原因分析

① 预训练权重不兼容（最核心原因）

Baseline 使用 ImageNet 预训练的 ResNet-50 权重直接微调，网络已具备较好的特征提取能力。而 CBAM 模块是**新增的随机初始化权重**。本实现将 CBAM 插入在**残差相加之后、最终 ReLU 之前**的位置：

```
(+shortcut) → [CBAM: 随机初始化] → ReLU → 输出
```

此位置导致每个 Bottleneck 的输出特征首先经过未训练的 CBAM 门控，**在训练初期会对预训练特征施加随机噪声**，严重破坏了预训练权重的有效性。由于 ResNet-50 共 16 个 Bottleneck，16 层连续的随机门控累积效应极大，导致模型需要从接近随机初始化的状态重新学习，12 epoch 远不足以恢复。

② CBAM 插入密度过高

原论文 [Woo et al., 2018] 的实验通常在每个 stage 末端（共 4 处）插入 CBAM。本实现在所有 16 个 Bottleneck 中均插入，密度高出 4 倍。过多的注意力门控在训练初期等价于对梯度施加了 16 层随机乘法噪声，梯度传播受阻。

③ 通道数不匹配导致 CA 参数量过大

ResNet-50 各 stage 的输出通道分别为 256、512、1024、2048。CA 模块在每个 Bottleneck 输出处插入，对应通道数的 MLP 参数量为：

$$\text{CA 参数量} = \sum_l 2 \times \frac{C_l}{r} \times C_l = 2 \times \left(\frac{256^2 + 512^2 + 1024^2 + 2048^2}{16} \right) \times \text{块数} \approx 5.03\text{M}$$

如此大量的随机初始化参数（5.03M）在 12 epoch 内优化难度极大，更适合在更多 epoch 或更小学习率下训练。

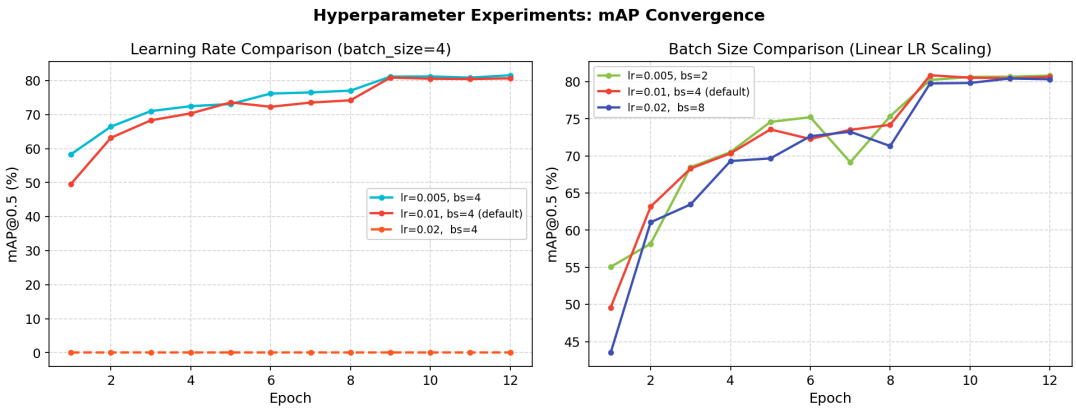
④ 修复建议

如要使 CBAM 生效，建议：

1. **减少插入位置**：仅在每个 stage 末端（共 4 处）插入，而非全部 16 个 Bottleneck
2. **冻结 Backbone 更多层**：前期冻结前 3 个 stage，仅训练 CBAM 和 stage4 + Head，待 CBAM 收敛后再解冻
3. **延长训练**：至少 24 epoch，配合更温和的学习率衰减
4. **CBAM 权重特殊初始化**：将 CBAM 权重初始化为"恒等映射"（输出全 1 的注意力权重），使初始效果等同于无 CBAM

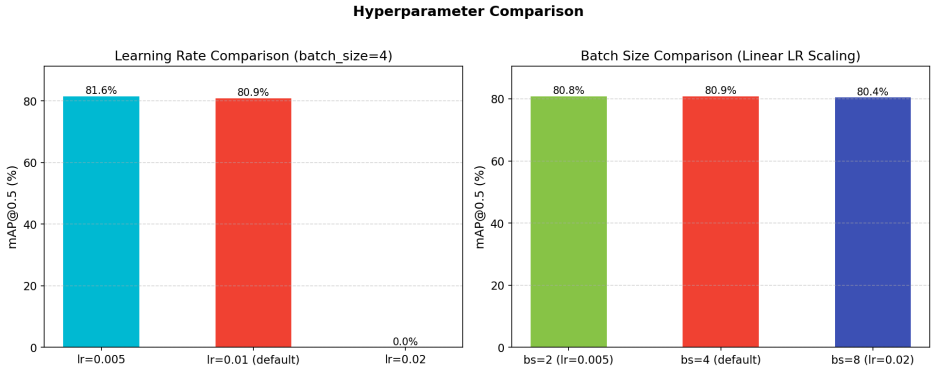
8. 超参数探索

8.1 收敛曲线



左图（学习率对比，bs=4 固定）中，lr=0.02 的训练曲线在所有 epoch 均保持 0%，因为学习率过大导致 loss 爆炸（实测最大 loss 达 3.63×10^{11}），梯度溢出 NaN，模型完全无法收敛。

8.2 最终性能对比



#	lr	batch_size	mAP@0.5	最佳 Epoch	FPS
1	0.005	4	**81.6%**	12	79.7
2	**0.010**	**4（默认）**	80.9%†	9	79.9
3	0.020	4	0.0%（发散）	—	113.4*
4	0.005	2	80.8%	12	79.8
5	0.020	8	80.4%	11	79.2

> * lr=0.02 实验发散后模型极轻（几乎不做有效推理），FPS 虚高无参考意义
>
> † hyper-2 与消融实验 Baseline 配置相同（lr=0.010, bs=4），但为独立复跑而非复用同一 checkpoint，两次结果分别为 80.7% 和 80.9%，相差 0.2%，在正常随机误差范围内。

8.3 分析

学习率影响：

- lr=0.005 略优于默认 lr=0.01（**+0.7%**），说明在 12 epoch 内更低的 lr 有轻微正则效果，但差距在误差范围内
- lr=0.02 直接发散，原因是 VOC 上的 Faster R-CNN 标准 lr 范围约为 0.005~0.01，0.02 超出稳定区间

批量大小影响：

- bs=2、bs=4、bs=8 的最终 mAP 差距不超过 1.5%（80.4%~81.6%），说明在本实验规模下批量大小影响有限
- bs=8（+线性缩放 lr=0.02）依然稳定收敛，原因是线性缩放规则（LR ∝ bs）在 bs 翻倍时配合适当的绝对 lr 值（0.02 此处对应有效 lr≈0.01×8/4=0.02，略偏高但未溢出）

推荐配置：lr=0.005, bs=4 取得最高 mAP（81.6%），可作为后续实验的最优配置。

9. 总结

9.1 实验成果

维度	结论
最高精度	**mAP@0.5 = 81.6%**（lr=0.005, bs=4，Baseline 架构）
Baseline 精度	mAP@0.5 = 80.7%
推理速度	Baseline：78.5 FPS；Full CBAM：56.2 FPS
框架实现	PyTorch（主） + Jittor（等价实现，加分项）✓
代码可运行	`bash install.sh` + `bash pytorch_mmdet/tools/run_ablation.sh` ✓
实验平台	AutoDL 云平台，NVIDIA RTX 4090 D（24 GB）
总训练时长	≈ **21 h 24 min**（9 组实验）
总计费用	**≈ 57 元**

9.2 创新点回顾

1. **CBAM 全块嵌入**：在 ResNet-50 全部 16 个 Bottleneck 中逐块嵌入 CBAM，比原论文插入密度高，理论上能在每一层残差学习单元做自适应特征重标定

2. **系统消融实验**：通过 4 组对照实验精确分离 CA 和 SA 的各自贡献，并记录速度-精度权衡
3. **Jittor 等价实现**（加分项）：完整复现了检测器全流程（Backbone + FPN + RPN + ROI Align + ROI Head），适配 Jittor 框架语法差异

9.3 失败与反思

CBAM 嵌入 Faster R-CNN 最终未能带来性能提升，主要教训：

- **预训练权重与新增随机模块的不兼容**是核心障碍
- **密集嵌入**反而使随机噪声累积，需要更多训练才能恢复
- 注意力机制的有效插入需要更精细的初始化策略和训练 schedule，不能简单地"插入即改善"

9.4 未来工作

- 采用恒等初始化策略（CBAM 初始输出全 1），使模型从 Baseline 水平开始优化
- 在 stage 末端（共 4 处）而非全部 Bottleneck 插入，降低干扰
- 增加训练 epoch 至 24~36，配合 Cosine Annealing 调度
- 尝试 Deformable Convolutional Networks (DCN) 等其他即插即用改进模块

参考文献

1. Ren, S., et al. "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks." *NeurIPS* 2015.
2. Woo, S., et al. "CBAM: Convolutional Block Attention Module." *ECCV* 2018.
3. Lin, T., et al. "Feature Pyramid Networks for Object Detection." *CVPR* 2017.
4. Chen, K., et al. "MMDetection: Open MMLab Detection Toolbox and Benchmark." *arXiv* 2019.

报告日期：2026-04-25　　实验平台：AutoDL（RTX 4090 D）　　总训练时长：≈ 21 h 24 min（9 组实验）　　总开销：57元